ISA 345 Final Project

May 8th, 2026
“Apex Event Solutions”

Spring 2026
Arthur Carvalho
Section A

Group Members:
Vicki Brandt
Dalton Kane
Eliana Ping
Jesse Rosner

Project Overview:

Apex Event Solutions is an event management and ticketing company that specializes in live events such as concerts and sporting events. The company currently relies on a poorly designed database that has resulted in multiple operational issues and informalities. Some issues include double booking of the venue and inefficient ticketing organization. In order to resolve these issues, we propose a relational database system that connects entities like events, venue, artist, customer, orders and more. This database will streamline business processes by connecting and organizing three key processes. First, the system will support purchasing tickets, which will also include customer payments and ticket information for each new order. Second, the system will register new events by linking them to specific artists and venues, as well as specific dates in order to prevent double booking. The last process is the payroll, which will compensate employees for their involvement for each live performance. This is a simplified description of the new database processes that will be further developed and shown throughout the rest of this document.

Database Design and Implementation:

Entities

Employees

- EmployeeID * - NUMBER (8)
- Name - VARCHAR (100)
- Dept - VARCHAR (100)
- EmpSalary - NUMBER (10 , 2)

This entity shows each employee working at the event management company. It identifies each employee by an ID which is the primary key. Then it also includes info about the name of the employee, their department, and their salary.

Orders

- OrderID * - NUMBER (8)
- Order_Date - DATE
- ShipDate - DATE
- EventID **FK** - NUMBER (8)
- CustomerID **FK** - NUMBER (8)

This entity has to do with individual orders made by consumers. Each order is identified by an ID. Info about the order also includes the date it was ordered and its ship date.

Tickets

- TicketID * - NUMBER (8)
- OrderID **FK** - NUMBER (8)
- Type - VARCHAR (100)

- EventID **FK** - NUMBER (8)

This entity has to do with individual tickets in every order. Each ticket has an ID and is connected to an OrderID. Each ticket has a type and is also connected to an event.

Payments

- PaymentID * - NUMBER (8)
- OrderID **FK** - NUMBER (8)
- PmtAmount - NUMBER (10,2)

This entity lists each payment received and associates it with an order.

Events

- EventID * - NUMBER (8)
- Type - VARCHAR (100)
- Event_Date - DATE
- Description - VARCHAR (1000)
- VenueID **FK** - NUMBER (8)
- ArtistID **FK** - NUMBER (8)

This entity has to do with specific events. Each event is identified by the EventID. Each event also has a type, date, and short description attached.

Artists

- ArtistID * - NUMBER (8)
- Name - VARCHAR (100)
- Genre - VARCHAR (150)

This entity has to do with the list of artists that the company has access to for performing at events. Each artist has a unique ID and their name and genre attached as well.

Venue

- VenueID * - NUMBER (8)
- City - VARCHAR (150)
- State - VARCHAR (50)
- Type (ex. Stadium, arena, concert hall) - VARCHAR (100)

This entity contains a list of potential venues. Each venue has an ID along with info about the type and location.

Customers

- CustomerID * - NUMBER (8)
- Name - VARCHAR (100)
- Type - VARCHAR (100)
- email - VARCHAR (100)

- Phone - VARCHAR (20)
- Address - VACHAR (150)

This entity includes a list of customers. Customer information such as email and phone is stored within the table for each customer.

EmpPayroll

- emppayrollID * - NUMBER (8)
- EmpID **FK** - NUMBER (8)
- HoursWorked - NUMBER (8,2)
- HourlyRate - NUMBER (8,2)
- AmtPaid - NUMBER (10,2)

This entity discusses the payroll for employees. Each payment given has an ID along with the details of the payment period.

Shifts

- EmployeeID * - NUMBER (8)
- EventID * - NUMBER (8)
- Duration - NUMBER (8,2)

This entity discusses the shifts worked by employees. There is a composite primary key made up of EmployeeID and EventID. There is also info included about the shift length

RELATIONSHIPS:

Employees → Events

Multiple employees work at one event

One employee works at many events

Events → Orders

Every order has one event

Each event has multiple orders

Events → artists

Each artist has many events

Each event can have multiple artists

Events → Venue

Each event has one venue

Each venue has multiple events

Orders → Payments

Each order can have multiple payments

Each payment can only have one order

Orders → Customers

Each order has one customer

Each customer has multiple orders

Orders → Tickets

Each order can have multiple tickets

Each ticket only had one order

Event → Tickets

Each ticket has one event

Each event has many tickets

EmpPayroll → Employee

Each employee has many paystub

Each payroll has one employee

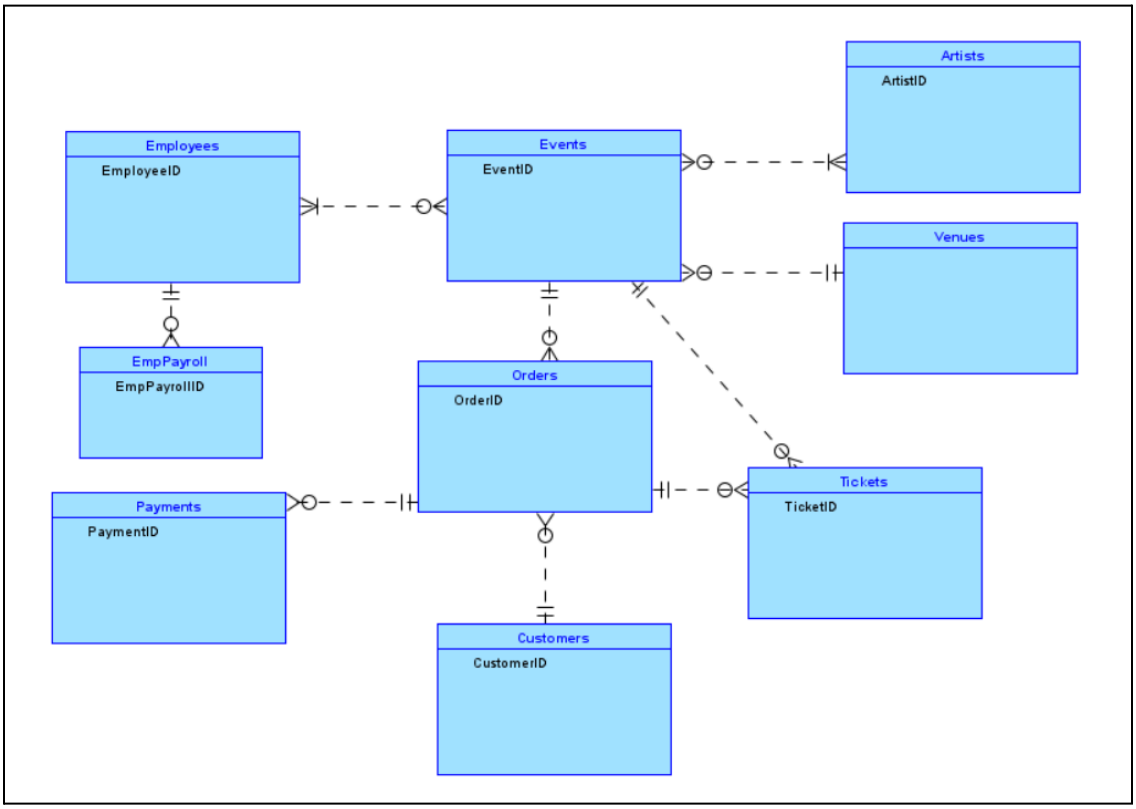
Shifts → Employee

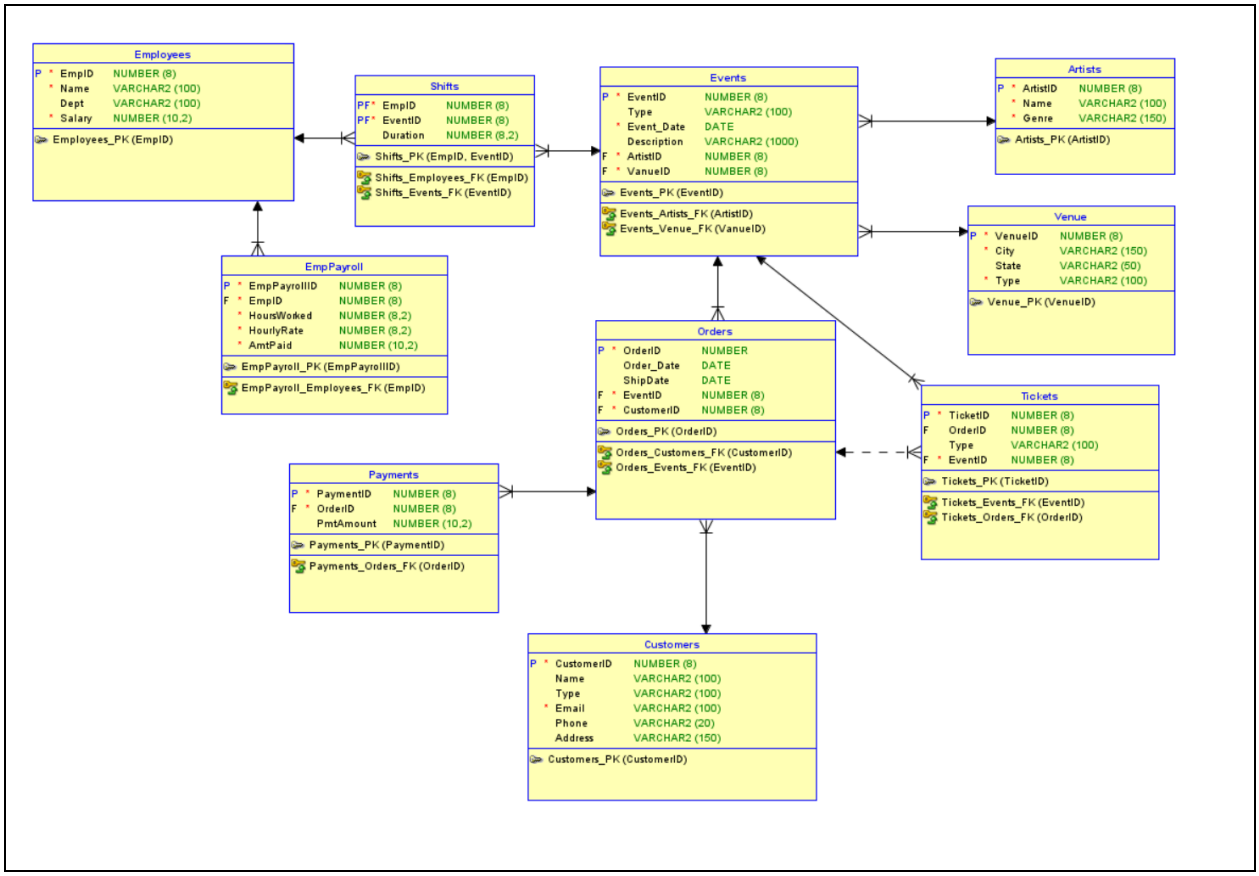
Each employee can work many shifts

Shifts → Event

Each event has many shift

Each shift has one event




Employees
P * EmpID NUMBER (8)
* Name VARCHAR2 (100)
Dept VARCHAR2 (100)
* Salary NUMBER (10,2)
Employees_PK (EmpID)
Shifts
PF* EmpID NUMBER (8)
PF* EventID NUMBER (8)
Duration NUMBER (8,2)
Shifts_PK (EmpID, EventID)
Shifts_Employees_FK (EmpID)
Shifts_Events_FK (EventID)
Events
P * EventID NUMBER (8)
Type VARCHAR2 (100)
* Event_Date DATE
Description VARCHAR2 (1000)
F * ArtistID NUMBER (8)
F * VanueID NUMBER (8)
Events_PK (EventID)
Events_Artists_FK (ArtistID)
Events_Venue_FK (VanueID)
Artists
P * ArtistID NUMBER (8)
* Name VARCHAR2 (100)
* Genre VARCHAR2 (150)
Artists_PK (ArtistID)
Venue
P * VenueID NUMBER (8)
* City VARCHAR2 (150)
State VARCHAR2 (50)
* Type VARCHAR2 (100)
Venue_PK (VenueID)
EmpPayroll
P * EmpPayrollID NUMBER (8)
F * EmpID NUMBER (8)
* HoursWorked NUMBER (8,2)
* HourlyRate NUMBER (8,2)
* AmtPaid NUMBER (10,2)
EmpPayroll_PK (EmpPayrollID)
EmpPayroll_Employees_FK (EmpID)
Orders
P * OrderID NUMBER
Order_Date DATE
ShipDate DATE
F * EventID NUMBER (8)
F * CustomerID NUMBER (8)
Orders_PK (OrderID)
Orders_Customers_FK (CustomerID)
Orders_Events_FK (EventID)
Tickets
P * TicketID NUMBER (8)
F OrderID NUMBER (8)
Type VARCHAR2 (100)
F * EventID NUMBER (8)
Tickets_PK (TicketID)
Tickets_Events_FK (EventID)
Tickets_Orders_FK (OrderID)
Payments
P * PaymentID NUMBER (8)
F * OrderID NUMBER (8)
PmtAmount NUMBER (10,2)
Payments_PK (PaymentID)
Payments_Orders_FK (OrderID)
Customers
P * CustomerID NUMBER (8)
Name VARCHAR2 (100)
Type VARCHAR2 (100)
* Email VARCHAR2 (100)
Phone VARCHAR2 (20)
Address VARCHAR2 (150)
Customers_PK (CustomerID)

Normalization Process Issues

When creating our model we kept normalization in mind and did not have many violations. First, our model automatically was in 1NF form because every cell contained one value. Second, we got our model into 2NF by ensuring that there were no partial dependencies and every column depended on the PK. Lastly, we had to put it into 3NF and we did have a couple functional dependencies that we had to fix. After resolving the anomalies, we resolved the following functional dependencies, resulting in the following:

1. VenueID → City, State, and Type
2. CustomerID → Name, Email, Phone, Address
3. EmpID → Dept, Salary

After identifying these functional dependencies we were able to normalize the model by removing anomalies and minimizing redundancy.

One note to keep in mind is that this model technically would not be in full 3NF due to a couple of small dependencies like the state depending on the city. As a group we decided that fixing these redundancies would be more redundant and not practical for use. It would lead for more unnecessary JOIN statements and such.

Operational SQL

Queries and Business Solutions

1. **Contain a subquery**

This query identifies employees whose base salary is higher than the company's overall average salary. This provides the HR and Finance departments with an immediate list of high-cost personnel. By monitoring these employees, management can review department budgets and ensure compensation aligns with operational value and the payroll transaction system.

```
SELECT name, dept, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

	NAME	DEPT	SALARY
1	John Smith	Management	75000
2	Emily Davis	Marketing	60000
3	Robert Downey	Management	90000
4	Scarlett J	Marketing	62000

2. **Contain an inner join**

This query generates a comprehensive "Event Master Schedule" that displays the event date, description, the performing artist's name, and the venue city. Since the events table only stores IDs (ArtistID, VenueID), this query translates those numbers into a readable format for event

coordinators and marketing teams. It fulfills the core business process of tracking new events by linking artists and venues seamlessly.

```
SELECT e.event_date, e.description, a.name AS artist_name, v.city, v.type AS venue_type
FROM events e
JOIN artists a ON e.artistid = a.artistid
JOIN venue v ON e.venueid = v.venueid
ORDER BY e.event_date;
```

	EVENT_DATE	DESCRIPTION	ARTIST_NAME	CITY	VENUE_TYPE
1	15-MAY-26	Summer Kickoff	The Midnight Echo	Austin	Amphitheater
2	20-MAY-26	Live at Nashville	Synthwave Dreams	Nashville	Club
3	01-JUN-26	The Big Apple Fest	Sarah Jenkins	New York	Arena
4	10-JUN-26	Acoustic Night	Velvet Underground Cover	Chicago	Theater
5	15-JUN-26	Neon Nights	DJ Quantum	Seattle	Outdoors
6	04-JUL-26	Independence Jazz	Blue Note Quartet	Denver	Ballroom
7	12-JUL-26	Metal Mania	Steel Panther	Miami	Beach Club
8	20-JUL-26	Pop Princess Tour	Luna and The Whalers	San Francisco	Hall
9	05-AUG-26	Country Roots	Cactus Jack	Atlanta	Stadium
10	15-AUG-26	Modern Synth Opera	Neon Skyline	Boston	Opera House

3. Contain an outer join

This query lists all tickets, specifically highlighting which tickets have not yet been assigned to an order (unsold tickets). This directly addresses your stated business problem of "missing out on potential revenue (under selling)". By using a LEFT OUTER JOIN between tickets and orders and filtering for records where the orderid IS NULL, the marketing team can immediately see exactly how much unsold inventory remains for upcoming events and launch targeted promotions.

```
SELECT t.ticketid, t.type AS ticket_type, e.description AS event_name, e.event_date
FROM tickets t
LEFT OUTER JOIN orders o ON t.orderid = o.orderid
JOIN events e ON t.eventid = e.eventid
WHERE t.orderid IS NULL;
```

	TICKETID	TICKET_TYPE	EVENT_NAME	EVENT_DATE
1	1041	GA	Summer Kickoff	15-MAY-26
2	1045	GA	Summer Kickoff	15-MAY-26
3	1042	VIP	Live at Nashville	20-MAY-26
4	1046	VIP	Live at Nashville	20-MAY-26
5	1043	GA	The Big Apple Fest	01-JUN-26
6	1044	Premium	Acoustic Night	10-JUN-26
7	1047	GA	Country Roots	05-AUG-26
8	1048	GA	Modern Synth Opera	15-AUG-26
9	1049	Premium	Modern Synth Opera	15-AUG-26
10	1050	GA	Modern Synth Opera	15-AUG-26

4. **Contain a GROUP BY clause without the HAVING clause**

This query calculates the total payroll expenses grouped by employee department. By joining the Employee and EmpPayroll tables and grouping by the dept attribute, management can see exactly how much money is being spent on Management, Security, Sound, etc. This supports the third primary transaction of our business, processing payroll, by providing high-level financial oversight.

```
SELECT e.dept, SUM(p.amtpaid) AS total_department_payroll
FROM employees e
JOIN emppayroll p ON e.empid = p.empid
GROUP BY e.dept;
```

	DEPT	TOTAL_DEPARTMENT_PAYROLL
1	Ticketing	1360
2	Marketing	2344
3	Security	3095
4	Management	3160
5	Operations	3100

5. **Contain a GROUP BY clause together with the HAVING clause**

This query identifies high-volume events by finding events that have sold more than 5 tickets in your database(for simplicity purposes we will assume 5 is a lot). Knowing which shows are the most popular allows management to adjust staffing levels for security and concessions, optimize future marketing budgets toward high-demand genres, or negotiate more lucrative sponsorship deals for similar upcoming dates.

```
SELECT  e.EventID, e.Description, COUNT(t.TicketID) AS TotalTicketsSold
FROM Events e
JOIN  Tickets t ON e.EventID = t.EventID
GROUP BY e.EventID, e.Description
HAVING COUNT(t.TicketID) > 5;
```

	EVENTID	DESCRIPTION	TOTALTICKETSSOLD
1	301	Summer Kickoff	8
2	303	The Big Apple Fest	6
3	304	Acoustic Night	6

6. **Define relevant INDEXES**

This index creates an index on the "Date" column in the events table. Event management companies constantly query their databases based on dates (e.g., finding all events happening

next week). An index on the date column will significantly speed up data retrieval times for the front-end application.

CREATE INDEX idx_event_date ON events(event_date);

7. **Define a relevant VIEW**

This view creates a VIP_Customer_Directory view that pulls the names, emails, and phone numbers of customers specifically listed as 'VIP' type. Instead of writing a query every time the marketing or customer service team needs to contact high-value clients for exclusive presales, they can simply SELECT * FROM VIP_Customer_Directory. This restricts access to raw table data while providing a quick, easy-to-read dashboard.

```
CREATE OR REPLACE VIEW vip_customer_directory AS
SELECT name, email, phone
FROM customers
WHERE type = 'VIP';
```

8. **Define a relevant TRIGGER**

This trigger prevents a double booking BEFORE INSERT on the events table. This directly solves the primary business problem listed in your project overview: "Double booking people". The trigger can be programmed to check if the proposed VenueID and "Date" already exist in the system. If they do, the trigger raises an application error and blocks the insert, guaranteeing a double booking can never happen at the database level.

```
CREATE OR REPLACE TRIGGER prevent_double_booking
BEFORE INSERT ON events
FOR EACH ROW
DECLARE
   v_count NUMBER;
BEGIN
   SELECT COUNT(*)
   INTO v_count
   FROM events
   WHERE venueid = :NEW.venueid
   AND event_date = :NEW.event_date;

   IF v_count > 0 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Error: The venue is already booked for this
date.');
   END IF;
```

END;
/

Deployment

SQL Agent:
User: TicketDatabase
Pass: M7amiUniversity
SQL Agent
This SQL Agent will assist the employees for aggregating and querying the data

Python based web application:
Application
This python based web application will assist customers with buying tickets and learning about events.

Conclusions & Lessons Learned

Overall, the proposed database streamlines business processes by connecting and organizing key functions. The first key process our database supports is ticket purchases, which is especially helpful for customers and managing the overall operations of the venues. Query 3 identifies the number of tickets sold and the number of tickets that are still available. This is helpful as customers are able to see how busy the event will be, and purchase any remaining tickets. This is also beneficial for management as they can identify which events are more popular than others. Query 5 also identifies high volume events, which aids in the reallocation of resources, like staff members, in order to maximize profits.

This database especially helps artists who are looking into the venue, as they don't need to worry about double booking, which is supported by Query 8. Query 8 utilizes a trigger to search for an existing VenueID and Date and will block a new insert from being added into the database. This means that no new bookings will be created if there is already one for a certain date or venue. This supports our second key process of the database, which is preventing double booking.

The last key process that is present within our model is the payroll process, which compensates employees for their involvement in each event. This is most helpful for employees and management. Query 1 identifies employees with a higher salary than the base salary, which can be used to identify employees who may be compensated for their hard work or high years of employment. This is helpful for management as they can better understand where they need to budget more and which employees they want to utilize for certain events. Query 4 is especially

helpful in this process as it identifies the expenses per employee departments. This identifies which departments may be most important to the business processes and which departments are receiving the most compensation. Query 5 also aids in the payroll process as it identifies events that may need more employees, which means that more employees will be paid for those events.

These three processes have been successfully streamlined through our databases and queries that we utilized above. Decision making is much easier as each of the queries identify key processes such as ticket purchasing, double booking errors, and employee compensation. Management is able to make more informed decisions and control certain attributes such as allocating employees and tickets to certain events.

A few decisions affected the overall design of our database, including the addition of a Ticket and EmployeePayroll table. We added these two tables as we knew it would help create a better understanding of how much is being spent on employees and how much can be earned through ticketing. This allows management to better allocate resources where they are needed most, which could include hiring or firing employees and selling more or less tickets in order to maximize profits. The only issues we faced when creating our database was understanding the relationships between our tables and how they would interact with one another. However, we discussed these issues with one another and added or deleted any tables that weren't beneficial to our process. For example, we added EmpPayroll and Tickets as we knew we could better understand the payments processes and aid the management of the venues, artists, customers, and more. This led us to our final database model which can be seen above.